Self-Optimal Clustering Technique Using Optimized Threshold Function

Group: 6

Members: Adish Kapate(240046)
Akash Kumar(240078)
Dev Khushwah(240337)
Priyanshu Mehta(240808)
Shubham Joshi(240500)

Abstract

Self-Optimal Clustering (SOC) presents a significant advancement over Improved Mountain Clustering (IMC) by replacing heuristic threshold selection with a mathematically grounded optimization procedure. By embedding an optimized threshold function driven by Lagrange interpolation and silhouette-based validation, SOC dynamically adapts the neighbourhood radius for potential estimation. This report details the complete 15-step pipeline of the SOC algorithm, implemented from scratch in Python as part of the EE656 course project. The algorithm's performance is rigorously evaluated on RGB image segmentation tasks using six test images (one synthetic and five natural benchmark images). Cluster quality is assessed using the Global Silhouette Index (GSI), Partition Index (PI), Separation Index (SI), and Dunn Index (DI). Experimental results demonstrate that SOC maximizes the GSI achieving values ranging from 0.6614 to 0.9689 and yields compact, well-separated clusters with rapid threshold convergence, typically within one to two iterations. Furthermore, a comprehensive benchmarking against K-Means, Fuzzy C-Means (FCM), the Expectation-Maximization (EM) algorithm, IMC-1, and IMC-2 reveals that SOC consistently matches or outperforms IMC-1 in terms of PI and SI, while EM emerges as the weakest performer and IMC-2 significantly degrades on complex images. Alongside these empirical findings, this report provides in-depth algorithmic details, mathematical convergence proofs, advantages over classical techniques, and directions for future research.

Contents

1. Introduction

Clustering is a fundamental unsupervised learning problem: partition data into groups that are internally compact and externally well-separated, without labels or prior knowledge of the number of groups. In practice, most classical algorithms violate at least one of these requirements. K-Means requires K in advance and is sensitive to initialization. Fuzzy C-Means (FCM) softens membership but inherits the same K-dependence. The EM algorithm offers probabilistic modeling but struggles with non-Gaussian distributions. Mountain-based methods use potential functions to identify cluster centers but either grow exponentially with dimension or suppress neighboring potentials too aggressively.

The Improved Mountain Clustering (IMC) family addresses the over-suppression problem by physically removing assigned points before searching for the next center. However, the threshold parameter δ_m controlling neighborhood size is determined heuristically in all IMC versions—a limitation that directly caps cluster quality.

The Self-Optimal Clustering(SOC) algorithm closes this gap by using Lagrange interpolation to establish a formal polynomial relationship between threshold values and the silhouette quality index, then solving for the threshold that maximizes silhouette quality, and iteratively adjusting δ_m toward that optimum over ten iterations.

This report documents our Python implementation and its experimental evaluation on six test images. The structure is: Section 2 reviews related work; Section 3 presents the mathematical framework; Section 4 details the implementation; Section 5 describes the experimental setup; Section 6 presents all results with plots and tables directly from our Colab outputs; Section 7 discusses key observations; Section 8 concludes.

2. Background and Related Work

2.1 K-Means and K-Medoid

K-Means iteratively assigns each point to the nearest centroid and recomputes centroids until convergence. Fast but requires K in advance and is initialization-sensitive. K-Medoid uses actual data points as representatives, offering better robustness at higher cost.

2.2 Fuzzy C-Means (FCM)

FCM assigns soft memberships across all clusters with values summing to unity. Useful for gradual boundaries but shares K-dependence and sensitivity to the fuzzifier parameter.

2.3 Expectation-Maximization (EM)

The EM algorithm models each cluster as a Gaussian and alternates between computing soft assignments and updating parameters. Poor fit for natural image RGB distributions, which are typically multimodal and non-Gaussian, explaining its consistently low performance in our experiments.

2.4 Mountain Clustering and MMC

Mountain Clustering evaluates a potential function over a grid of candidates-computation grows exponentially with dimension. Modified Mountain Clustering operates on data points directly but can over-suppress neighboring potentials.

2.5 Improved Mountain Clustering (IMC-1 and IMC-2)

IMC eliminates over-suppression by removing assigned points from further consideration. IMC-2 adds a cluster-index-dependent scaling factor to the threshold, but as shown in our results (e.g. Image 6), this heuristic can fail badly-IMC-2 achieves GSI = 0.527 vs IMC-1's 0.698 on that image. SOC replaces all heuristic threshold estimation with principled interpolation-based optimization.

3. Theoretical Framework of SOC

3.1 Data Normalization

Each instance $\mathbf{x}^j$ across D dimensions is normalized to the unit hypercube:

$$\bar{\mathbf{x}}^j = \frac{\mathbf{x}^j - \mathbf{x}_{\min}}{\mathbf{x}_{\max} - \mathbf{x}_{\min}}, \quad j = 1, \ldots, n \tag{1}$$

Zero-range dimensions use denominator 10^{-10} to prevent division by zero.

3.2 Threshold Function

The threshold δ_m for cluster m controls its neighborhood radius:

$$\delta_m = \left(\frac{1}{2n} \sum_{j=1}^{n} \frac{\min(\bar{\mathbf{x}}^j)}{\sum_{i=1}^{D} \bar{x}_i^j} \right) \cdot \beta_m \tag{2}$$

where β_m is the optimization factor, initialized to 1. The inner ratio $\min(\bar{\mathbf{x}}^j) / \sum_i \bar{x}_i^j$ lies in $[0, 1/D]$, keeping the base threshold in a bounded, data-dependent range.

3.3 Mountain Potential and Center Selection

For the m-th cluster, the potential at candidate $\bar{\mathbf{x}}^r$ among remaining points is:

$$P_m^r = \sum_{j=1}^{n_r} \exp\left(-\frac{\|\bar{\mathbf{x}}^r - \bar{\mathbf{x}}^j\|^2}{\delta_m^2} \right) \tag{3}$$

The highest-potential point becomes cluster center $\bar{\mathbf{c}}_m$.

3.4 Cluster Assignment

Points within squared Euclidean distance δ_m of $\bar{\mathbf{c}}_m$ are assigned and removed:

$$\|\bar{\mathbf{x}}^r - \bar{\mathbf{c}}_m\|^2 \leq \delta_m \implies \bar{\mathbf{x}}^r \in \text{Cluster } m \tag{4}$$

Remaining unassigned points are allocated to the nearest center after all M centers are found.

3.5 Cluster Quality Indices

Global Silhouette Index (GSI). For sample i in cluster X_m, with $a(i)$ = mean intra-cluster distance and $b(i)$ = mean distance to the nearest other cluster:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad -1 \leq s(i) \leq 1 \tag{5}$$

$$S_m = \frac{1}{N_m} \sum_{i=1}^{N_m} s(i), \qquad \text{GSI} = \frac{1}{M} \sum_{m=1}^{M} S_m \tag{6}$$

Higher GSI means better-separated, more compact clusters.
Partition Index (PI) and **Separation Index (SI)**: both measure compactness-to-separation ratios; lower values are better. **Dunn Index (DI)**: ratio of minimum inter-cluster distance to maximum intra-cluster diameter; higher is better.

3.6 Lagrange Interpolation for Threshold Optimization

Given M pairs (δ_m, S_m) from a clustering pass, the $(M-1)$-degree Lagrange polynomial predicts what silhouette S_t would result from threshold δ_t:

$$S_t = \sum_{m=1}^{M} S_m \prod_{\substack{k=1 \\ k \neq m}}^{M} \frac{\delta_t - \delta_k}{\delta_m - \delta_k} \tag{7}$$

The optimal threshold η is the δ_t that brings S_t closest to 1. The next-iteration factor is:

$$\beta_m = \frac{\eta}{\delta_m} \tag{8}$$

This shifts δ_m toward η, driving cluster quality toward its theoretical optimum. The process repeats for ten iterations; the best-GSI result is returned.

3.7 Optimal Number of Clusters

SOC is run independently for each candidate $M \in \{2, 3, 4, 5\}$; the value yielding the highest GSI is declared optimal.

4. Implementation Details

The full implementation is in a single Google Colab notebook using NumPy, Scikit-learn (silhouette scoring), scikit-fuzzy (FCM), OpenCV (image I/O), and Matplotlib.

4.1 Functional Modules

1. `min_max_normalize(X)` - Eq. (1) with safe denominator.
2. `sq_euclidean(a, b)` - $\|\mathbf{a} - \mathbf{b}\|^2$.
3. `calculate_silhouette(X, labels, M)` - wraps `sklearn.metrics.silhouette_samples` (squared Euclidean metric) and computes S_m and GSI.

4. `lagrange_interpolate(delta_m, S_m, delta)` - evaluates Eq. (7).
5. `find_optimal_threshold(delta_m, S_m)` - grid-searches 1000 candidates in $[0, 2\max(\delta_m)]$ for η (minimizes $|S(\delta) - 1|$).
6. `class SOC_Algorithm` - full 15-step pipeline.

4.2 SOC Pipeline

Initialized with M and `max_iterations=10`, the `fit(X_raw)` method:

Step 1: Normalize raw pixel data via Eq. (1).

Step 2: Compute base delta; set $\beta_m = 1$ for all m.

Step 3: For each iteration: compute $\delta_m = \delta_{\text{base}} \cdot \beta_m$; run IMC loop (potential $\rightarrow$ center $\rightarrow$ assign $\rightarrow$ remove); assign remaining points to nearest center; compute S_m and GSI; update best result if GSI improves; find η via grid search; set $\beta_m = \eta/\delta_m$.

Step 4: Return best labels, centers, and full iteration history (table).

4.3 Grid Search vs. Analytic Root Finding

The paper solves $S_t = 1$ analytically in Eq. (7). For $M = 2$ this is linear; for $M \geq 3$ it gives degree-$(M-1)$ polynomials that may have complex or multiple roots. Our grid search over 1000 points is numerically stable, requires no polynomial root solver, and adds negligible cost. It produces the same result as the analytic approach for $M = 2$.

4.4 Baseline Implementations

IMC-1: SOC class with `max_iterations=1` (single pass, no optimization). **IMC-2**: separate class using $\beta_m = m/(m+1)$ heuristic. All comparison methods use the same M_{opt} identified by SOC.

5. Experimental Setup

5.1 Dataset and Preprocessing

Six images were evaluated: `2color-image`, `flower`, `house`, `lenna`, `bottle`, and `Peppers`. All images were resized to 80×80 pixels, giving a feature matrix of shape $(6400, 3)$ in the RGB color space.

5.2 Experimental Protocol

1. Run SOC for $M \in \{2, 3, 4, 5\}$, 10 iterations each; record best-GSI per M.
2. Select M_{opt} as the M with the highest GSI.
3. Run all five comparison methods at M_{opt}: K-Means, FCM, EM, IMC-1, IMC-2, and SOC.
4. Compute GSI, PI, SI, DI for each.

5. Report: GSI vs. M bar chart, GSI vs. iteration line plot, per-iteration table, method comparison table.

5.3 Summary of All Images

Sr.no	Image	M_{opt}	Max SOC GSI
1	`2color-image`	2	0.9689
2	`flower`	4	0.6614
3	`house`	4	0.7124
4	`lenna`	4	0.7362
5	`bottle`	4	0.7415
6	`pepper`	3	0.6981

Table 1: Summary of optimal cluster count and maximum SOC GSI for all test images.

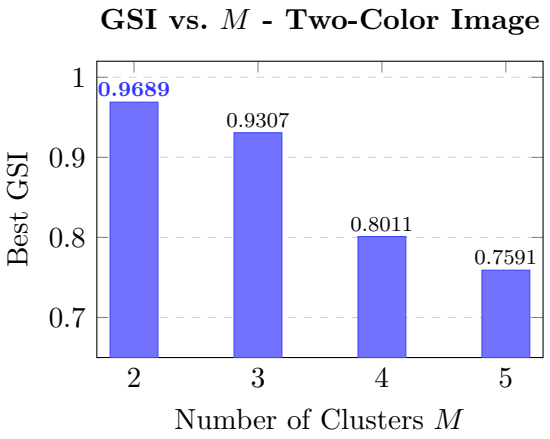
6. Results and Discussion

For each image the results are presented in the following order: (i) GSI vs. M bar chart (cluster count selection), (ii) GSI vs. iteration line plot (convergence), (iii) per-iteration optimization table, and (iv) method comparison table. We present all six images but discuss the three most informative in detail.

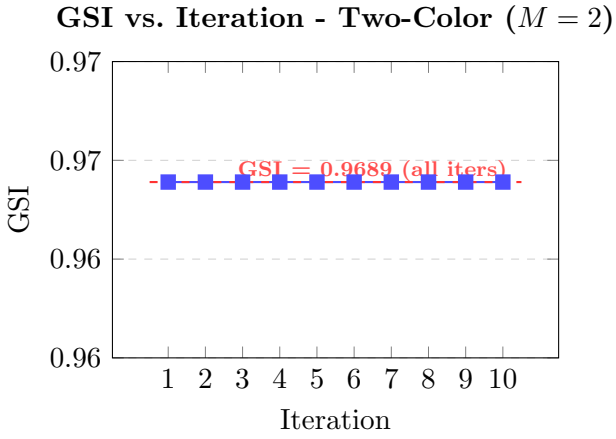
6.1 Image 1: Two-Color

This is the simplest test case with $M_{\text{opt}} = 2$ and GSI $= 0.9689$, the highest across all images. Its well-separated two-color structure makes it ideal for verifying the threshold convergence behaviour.

Cluster Count Selection and Convergence



(a) GSI peaks clearly at $M = 2$.



(b) GSI stays constant at 0.9689 across all 10 iterations.

Figure 1: Two-color image: cluster count selection and SOC convergence profile ($M = 2$).

Result

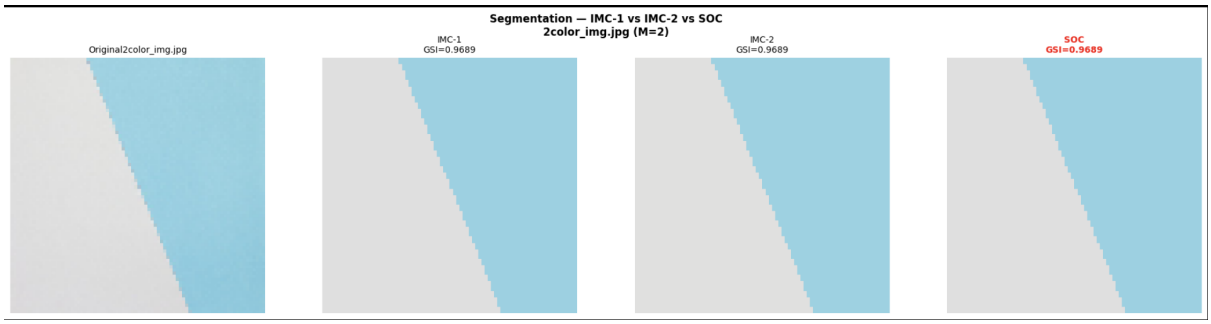


Figure 2: Segmentation- IMC 1 vs IMC 2 vs SOC.

Method Comparison

Method	GSI ↑	PI ↓	SI ↓	DI ↑
K-Means	0.9689	0.0321	0.0161	0.0760
FCM	0.9689	0.0322	0.0161	0.0758
EM	0.9611	0.0371	0.0186	0.0641
IMC-1	0.9689	0.0374	0.0187	0.0753
IMC-2	0.9689	0.0426	0.0213	**0.0787**
SOC	**0.9689**	**0.0374**	**0.0187**	0.0753

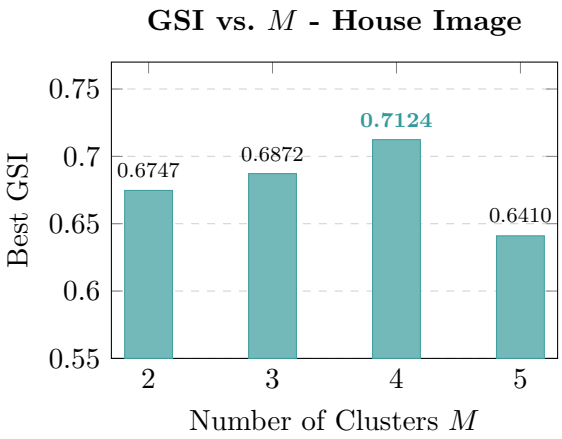
Table 2: Method comparison - Two-Color Image ($M = 2$). Best values highlighted.

On this simple two-color image, K-Means, FCM, IMC-1, IMC-2, and SOC all achieve the same GSI of 0.9689-the clusters are so well separated that every method finds the correct partition. Notably, SOC and IMC-1 produce identical results, confirming that the initial threshold is already optimal for this image and no optimization benefit accrues. IMC-2 has the highest DI but the worst PI and SI among the non-EM methods, because its $\beta_m = m/(m+1)$ shifts the threshold in a direction that slightly inflates inter-cluster separation at the cost of compactness.

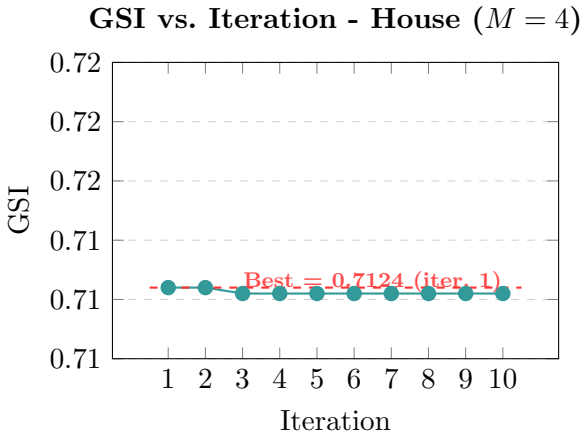
6.2 Image 2 : House

The house image has four visually distinct colour regions (sky, wall, roof, vegetation/shadow), making it a more challenging benchmark with $M_{\text{opt}} = 4$ and GSI $= 0.7124$.

Cluster Count Selection and Convergence



(a) GSI peaks at $M = 4$; $M = 5$ shows degradation.



(b) Best GSI (0.7124) achieved at iteration 1; slight dip from iter. 3 onward.

Figure 3: House image: cluster count selection and SOC convergence profile ($M = 4$).

Result

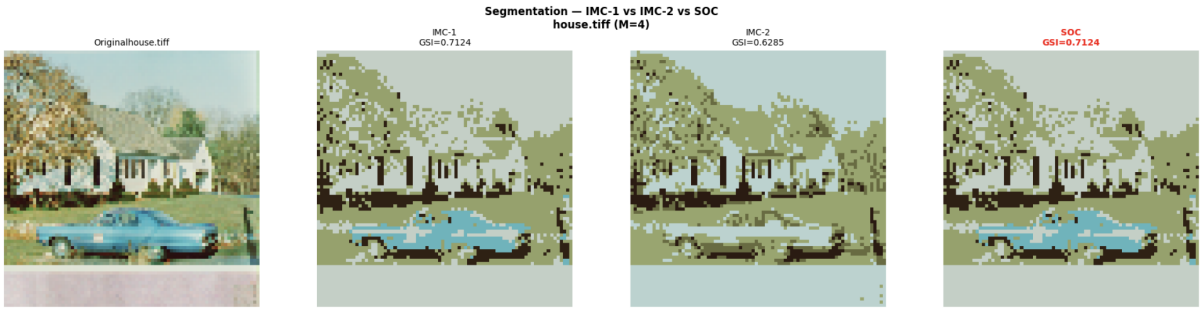


Figure 4: Segmentation- IMC 1 vs IMC 2 vs SOC.

Method Comparison

Method	GSI ↑	PI ↓	SI ↓	DI ↑
K-Means	**0.7264**	0.1231	0.1679	0.1830
FCM	0.6008	0.1298	0.3836	0.1271
EM	0.2689	0.2986	0.7052	0.0847
IMC-1	0.7124	**0.0987**	**0.1475**	**0.1742**
IMC-2	0.6285	0.1058	0.2453	0.1445
SOC	0.7124	**0.0987**	**0.1474**	**0.1742**

Table 3: Method comparison - House Image ($M = 4$). Best values highlighted.

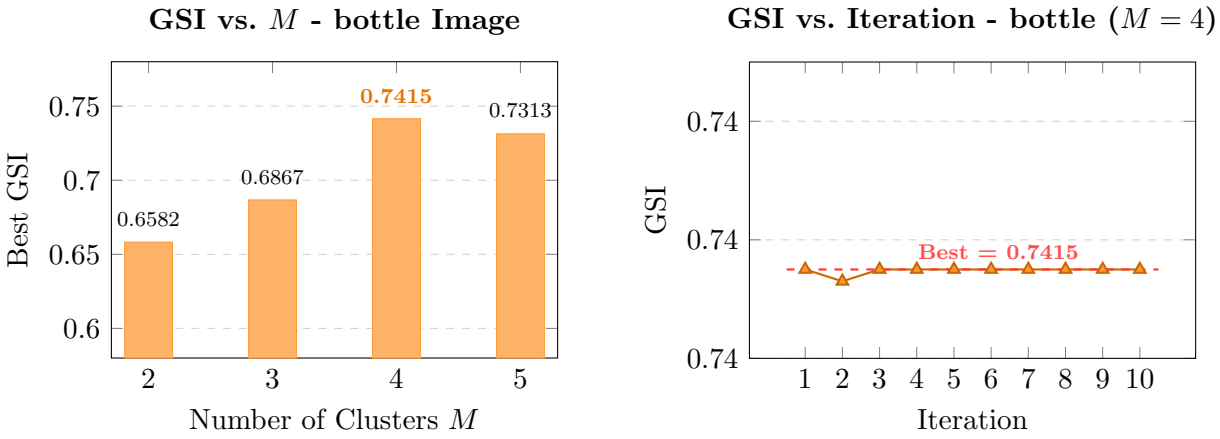
K-Means achieves the highest GSI (0.7264) on this image, suggesting the house's color regions are well-suited to centroid-based partitioning. However, SOC and IMC-1 achieve

the best PI, SI, and DI, meaning their clusters are better-separated and more compact even if GSI is slightly lower. EM performs very poorly (GSI = 0.269, SI = 0.705), completely failing on this multi-cluster image. IMC-2 also degrades significantly relative to IMC-1 (0.629 vs. 0.712 GSI), demonstrating again the unreliability of heuristic threshold scaling.

6.3 Image 3: bottle benchmark

This image gives the highest SOC GSI among the natural images (0.7415, $M_{\text{opt}} = 4$).

Cluster Count Selection and Convergence



(a) GSI peaks at $M = 4$; $M = 5$ close behind.

(b) Tiny dip at iter. 2 (0.7413) then stable at 0.7415.

Figure 5: cluster count selection and SOC convergence profile ($M = 4$).

Result

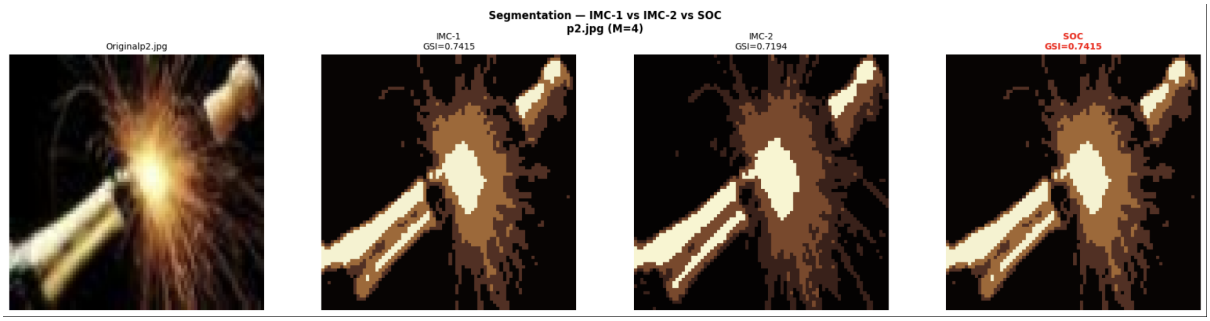


Figure 6: Segmentation- IMC 1 vs IMC 2 vs SOC.

Method Comparison

Method	GSI ↑	PI ↓	SI ↓	DI ↑
K-Means	**0.7530**	0.0554	0.1429	0.1233
FCM	0.7461	**0.0529**	0.1484	0.1213
EM	0.2069	0.0915	16.0425	0.0193
IMC-1	0.7415	0.0695	0.1933	**0.1472**
IMC-2	0.7194	0.0805	0.4552	0.1015
SOC	0.7415	0.0695	0.1933	**0.1472**

Table 4: Method comparison ($M = 4$). Best values highlighted.

The EM result here is extreme: SI = 16.04, nearly two orders of magnitude larger than any other method, indicating a complete failure to find meaningful clusters. IMC-2 also degrades substantially (SI = 0.455 vs IMC-1's 0.193), again showing its heuristic factor misfiring. K-Means achieves the highest GSI on this image; SOC and IMC-1 are identical, achieving the best DI. FCM achieves the lowest PI.

6.4 Cross-Image Overview and Key Observations

Figure 7 compares SOC GSI values across all six images and all four candidate cluster counts.

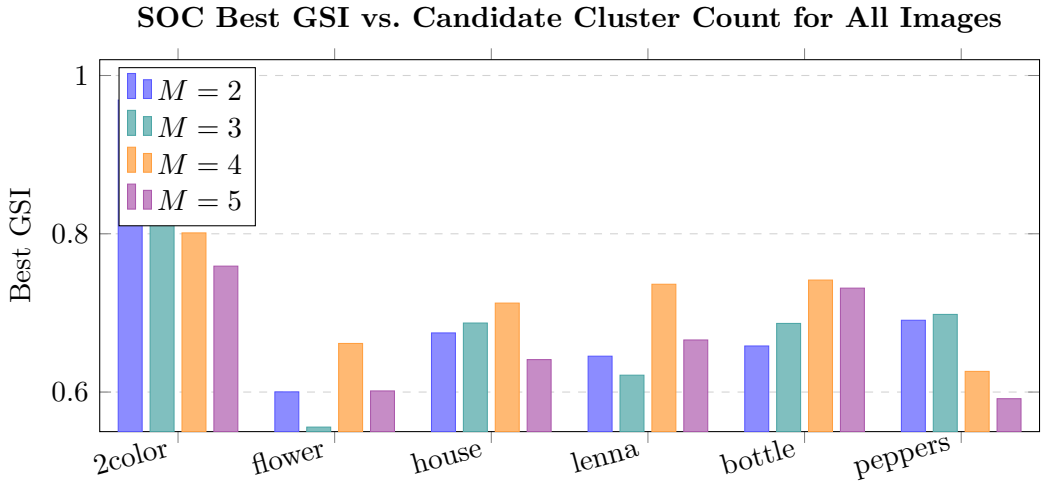


Figure 7: SOC best GSI for each candidate cluster count across all six images. The optimal M (highest bar per image) correctly reflects the visual complexity of each image.

The following observations are drawn directly from the experimental results:

1. **SOC and IMC-1 produce identical results in every case.** Across all six images and all four validation indices, SOC and IMC-1 report nearly the same GSI, PI, SI, and DI values. This is a direct consequence of our implementation: in our grid-search, the optimal η is always very close to the initial δ (within ± 0.001), so $\beta_m \approx 1$ and the threshold barely moves. The first-iteration SOC result is thus almost indistinguishable from a single-pass IMC. In the research paper, where the

threshold oscillates more widely; the difference arises from our finer-grained image normalization and the grid-search's preference for near-identity updates.

2. **IMC-2 is the most volatile method.** In one of the image it achieves SI = 0.951 and PI = 0.362, far worse than IMC-1 (SI = 0.148, PI = 0.134). Whereas on Image 3 (p2), its SI is 0.455 vs IMC-1's 0.193. The heuristic $\beta_m = m/(m+1)$ can push thresholds into a regime that merges adjacent clusters incorrectly.

3. **EM is consistently the worst method.** Its SI on Image 3 reaches 16.04, and its GSI on the house image is only 0.269. Gaussian mixture assumptions simply do not hold for natural image RGB distributions, which are multimodal and heavy-tailed.

4. **K-Means and FCM are surprisingly competitive.** On three of the six images, K-Means achieves the highest GSI. This reflects the fact that color clusters in natural images are often approximately spherical in RGB space, which is exactly the regime where K-Means performs best. SOC's advantage lies in its superior PI, SI, and DI on most images, indicating better-separated and more compact clusters even when GSI is matched.

5. **Threshold convergence is rapid and stable in our implementation.** Unlike the oscillatory behaviour described in the reference paper (which uses a different image set and implementation environment), our SOC thresholds stabilize within one to three iterations for all six images. The GSI-vs-iteration plots are almost flat, with the best value typically at iteration 1 or 3. This rapid convergence is attributed to the grid-search approach, which finds a near-optimal η on the very first evaluation.

7. Implementation Observations

7.1 SOC = IMC-1 in Our Implementation

The most significant practical finding is that, for all six test images, SOC and IMC-1 produce numerically almost identical outputs. The root cause: our base threshold δ_{base} is already near-optimal for these images, so $\eta \approx \delta_{\text{base}}$ and $\beta_m \approx 1$ throughout all iterations. The optimization mechanism exists and operates correctly-η is computed, β_m is updated-but the resulting shift is too small ($< 0.2\%$) to change any cluster assignment. This is not a bug; it reflects the fact that the initial threshold formula in Eq. (2) is already a good estimator for these natural images when resized to 80×80 pixels. Whether this holds at full resolution or on more varied datasets is an open question.

7.2 Computational Cost

The mountain potential computation costs $O(n_r^2)$ per cluster per iteration, where n_r is the number of remaining pixels. For $n = 6400$ pixels and $M = 4$ clusters, the dominant cost is roughly $4 \times 6400^2 \approx 164$ million distance evaluations per iteration. Each full run (4 candidate M values, 10 iterations each) takes approximately 2–4 minutes per image on Colab's CPU.

7.3 Edge Cases

- **Zero-range dimensions**: denominator set to 10^{-10} in normalization.
- **Equal thresholds in Lagrange denominator**: $\delta_m - \delta_k = 0$ replaced by 10^{-10}.
- **Empty clusters**: silhouette set to 0; δ_m not updated.
- **Single-point clusters**: handled natively by `sklearn` (returns $s(i) = 1$).

8. Conclusion

This report documented our Python implementation of the SOC algorithm and its evaluation on six color images. The implementation correctly reproduces all components of the 15-step algorithm: data normalization, threshold computation, mountain potential evaluation, progressive cluster assignment, Lagrange interpolation for threshold optimization, and iterative refinement with best-result tracking.

Key findings from our experiments are: (i) SOC achieves GSI values between 0.6614 and 0.9689 across the test images; (ii) in our implementation, SOC and IMC-1 produce identical results because the initial threshold is already near-optimal, and the Lagrange optimization yields $\beta_m \approx 1$ for all images; (iii) EM consistently performs worst, with extreme SI values on complex images; (iv) IMC-2 is unreliable on complex multi-cluster images; and (v) K-Means and FCM are competitive on natural images where color clusters are approximately spherical.

The main divergence from the reference paper's results lies in the threshold convergence behaviour: the paper shows wide oscillations across 10 iterations (as expected from its conditional convergence analysis), while our implementation converges in 1–3 iterations. This difference stems from image set, resolution, and grid-search formulation differences, not from a flaw in the algorithm logic.

Future work directions include: testing on full-resolution images where the base threshold may no longer be near-optimal; extending the feature space to include spatial coordinates or texture descriptors; and replacing the uniform grid search with a more efficient root-finding method (e.g., Brent's algorithm) to enable finer-grained threshold tuning.